



دانشگاه تهران

پردیس دانشکده های فنی

دانشکده مهندسی برق و کامپیوتر

گزارش بخش RPC

عنوان درس: مبانی رایانش توزیع شده

امین آقاکثیری – 810101381

مهدی نائینی – 810101536

بهار 1405

مقدمه

هدف از این بخش تمرین، بررسی مفهوم RPC و نقش آن در طراحی سیستم های توزیع شده است. در سیستم های توزیع شده، بخش های مختلف یک برنامه معمولا روی ماشین ها یا سرویس های جداگانه اجرا می شوند و لازم است این بخش ها بتوانند با یکدیگر ارتباط برقرار کنند. یکی از روش های مهم برای این ارتباط، استفاده از RPC یا Remote Procedure Call است.

تعریف RPC

RPC مخفف Remote Procedure Call به معنای فراخوانی تابع از راه دور است. در این روش، یک برنامه که معمولا نقش کلاینت را دارد، تابعی را فراخوانی می کند که در واقع روی یک برنامه یا سرویس دیگر و در یک ماشین دیگر اجرا می شود. این ماشین دیگر معمولا نقش سرور را دارد.

در فراخوانی معمولی یک تابع، برنامه تابع را در همان فضای حافظه و همان process اجرا می کند. اما در RPC ، تابع مورد نظر در یک process دیگر و معمولا روی یک ماشین دیگر قرار دارد. بنابراین برای اجرای آن لازم است اطلاعات ورودی تابع از طریق شبکه به سرور ارسال شود و نتیجه اجرای تابع نیز دوباره از طریق شبکه به کلاینت برگردد.

هدف اصلی RPC این است که این فرآیند تا حد امکان شبیه فراخوانی تابع معمولی به نظر برسد. برای مثال، کلاینت ممکن است تابعی مانند Login(username, password) را صدا بزند. اما این تابع در واقع درخواست را به سرور احراز هویت ارسال می کند، سرور آن را پردازش می کند و نتیجه موفق یا ناموفق بودن ورود را برمی گرداند.

ایده اصلی فراخوانی تابع از راه دور

ایده اصلی در RPC این است که ارتباط شبکه ای بین دو سرویس در قالب فراخوانی تابع مدل شود. یعنی به جای این که برنامه نویس مستقیما socket باز کند، پیام شبکه ای بسازد، داده ها را ارسال کند، پاسخ را بخواند و آن را تفسیر کند، یک تابع معمولی را صدا می زند و سیستم RPC پشت صحنه تمام این مراحل را انجام می دهد.

برای مثال فرض می کنیم سرویس وب در VM1 قرار دارد و سرویس احراز هویت در VM2 اجرا می شود. کاربر در صفحه وب نام کاربری و رمز عبور خود را وارد می کند. سرویس وب نباید خودش فایل کاربران را بخواند یا

احراز هویت را انجام دهد. به جای آن، یک فراخوانی RPC به سرویس احراز هویت ارسال می کند. این فراخوانی شامل username و password است. سرویس احراز هویت در VM2 داده ها را بررسی کرده و نتیجه را به VM1 برمی گرداند.

در ظاهر، کد سمت وب ممکن است شبیه این باشد:

```
result = AuthService.Login(username, password)
```

اما در واقع پشت این خط کد، داده ها به پیام شبکه ای تبدیل می شوند، از VM1 به VM2 فرستاده می شوند، در VM2 پردازش می شوند و پاسخ دوباره به VM1 بازگردانده می شود.

تفاوت فراخوانی محلی و فراخوانی راه دور

فراخوانی محلی زمانی اتفاق می افتد که تابع مورد نظر در همان برنامه، همان فرآیند و همان فضای حافظه اجرا شود. در این حالت، ارسال پارامترها و دریافت نتیجه بسیار سریع است، زیرا همه چیز داخل یک برنامه انجام می شود. اگر تابع خطا داشته باشد، معمولا خطا مربوط به منطق برنامه، مقدار ورودی یا وضعیت حافظه همان برنامه است.

اما در فراخوانی راه دور، تابع در یک سیستم دیگر اجرا می شود. در نتیجه چند تفاوت مهم به وجود می آید. اولین تفاوت مربوط به شبکه است. در RPC ، درخواست و پاسخ باید از طریق شبکه منتقل شوند. بنابراین تاخیر شبکه، قطع شدن ارتباط، timeout، از دست رفتن پیام یا در دسترس نبودن سرور ممکن است رخ دهد. این خطاها در فراخوانی محلی وجود ندارند.

تفاوت دوم مربوط به سرعت است. فراخوانی محلی معمولا بسیار سریع است، اما فراخوانی راه دور به دلیل serialization، ارسال داده در شبکه، پردازش در سرور و دریافت پاسخ زمان بیشتری نیاز دارد. بنابراین نباید با RPC مانند یک تابع محلی بسیار ارزان و سریع رفتار کرد.

تفاوت سوم مربوط به داده ها است. در فراخوانی محلی، پارامترها در همان حافظه برنامه قرار دارند، اما در RPC باید پارامترها به فرمتی قابل ارسال در شبکه تبدیل شوند. به این فرآیند serialization گفته می شود. در سمت دیگر نیز داده دریافتی باید دوباره به ساختار قابل استفاده در برنامه تبدیل شود که به آن deserialization گفته می شود.

نقش Client Stub و Server Stub

یکی از اجزای مهم درRPC ، stub است. Stub مانند یک واسطه عمل می کند و باعث می شود فراخوانی راه دور شبیه فراخوانی محلی دیده شود.

Client Stub در سمت کلاینت قرار دارد. وقتی برنامه کلاینت تابع راه دور را صدا می زند، در واقع ابتدا Client Stub فراخوانی می شود. Client Stub پارامترهای ورودی را دریافت می کند، آن ها را به قالب مناسب تبدیل می کند، پیام درخواست را می سازد و آن را از طریق شبکه به سرور ارسال می کند. سپس منتظر پاسخ سرور می ماند، پاسخ را دریافت می کند، آن را به شکل قابل استفاده برای برنامه تبدیل می کند و نتیجه را به برنامه کلاینت برمی گرداند.

Server Stub در سمت سرور قرار دارد. وقتی پیام درخواست از شبکه به سرور می رسد، Server Stub آن را دریافت می کند. سپس داده های داخل پیام را استخراج و deserialization می کند. بعد تابع واقعی سمت سرور را با پارامترهای مناسب صدا می زند. پس از اجرای تابع، نتیجه را دریافت می کند، آن را serialization می کند و به سمت کلاینت می فرستد.

بنابراین Client Stub و Server Stub باعث پنهان شدن جزئیات شبکه می شوند. برنامه نویس در سمت کلاینت احساس می کند یک تابع عادی را صدا زده است و برنامه نویس در سمت سرور نیز تابع واقعی را پیاده سازی می کند، بدون این که لازم باشد درگیر تمام جزئیات ارسال و دریافت پیام شود.

مفهوم Serialization و Deserialization

در RPC داده ها باید از یک برنامه به برنامه دیگر و معمولا از یک ماشین به ماشین دیگر ارسال شوند. اما داده های داخل برنامه به شکل ساختارهای حافظه ای هستند و مستقیما قابل ارسال از طریق شبکه نیستند. برای حل این مشکل از serialization استفاده می شود.

Serialization یعنی تبدیل داده ها و ساختارهای برنامه به یک قالب قابل انتقال یا ذخیره سازی. برای مثال یک شی شامل username و password می تواند به JSON ، XML یا Protocol Buffers تبدیل شود. پس از این تبدیل، داده به صورت یک رشته یا مجموعه ای از بایت ها قابل ارسال از طریق شبکه خواهد بود.

Deserialization فرآیند برعکس است. یعنی وقتی سرور پیام را دریافت می کند، باید آن را از قالب انتقالی مثل JSON یا بایت های Protocol Buffers به ساختار قابل استفاده در برنامه تبدیل کند.

برای مثال در یک سیستم احراز هویت، کلاینت ممکن است داده زیر را ارسال کند:

```
{
  "username": "alice",
  "password": "alice123"
}
```

این داده در سمت کلاینت serialization شده و از طریق شبکه ارسال می شود. سپس در سمت سرور deserialization می شود تا برنامه بتواند username و password را به عنوان متغیرهای عادی پردازش کند.

فرمت serialization تاثیر زیادی بر کارایی و سادگی سیستم دارد.JSON ساده و قابل خواندن است، اما نسبت به فرمت های باینری مانند Protocol Buffers حجم بیشتری دارد و کندتر پردازش می شود. در مقابل، Protocol Buffers سریع تر و کم حجم تر است، اما خوانایی مستقیم آن برای انسان کمتر است و معمولا به تعریف schema نیاز دارد.

مفهوم IDL

IDL مخفف Interface Definition Language است و به معنای زبان تعریف واسط می باشد. در بسیاری از فناوری های RPC ، قبل از پیاده سازی کلاینت و سرور، باید interface سرویس به صورت دقیق تعریف شود. این interface مشخص می کند که سرویس چه تابع هایی دارد، هر تابع چه ورودی هایی می گیرد و چه خروجی ای برمی گرداند.

برای مثال در gRPC معمولا از فایل proto استفاده می شود. در این فایل سرویس ها و پیام ها تعریف می شوند. نمونه ساده آن برای سرویس احراز هویت می تواند به شکل زیر باشد:

```
service AuthService {
  rpc Login(LoginRequest) returns (LoginResponse);
}
```

در این تعریف مشخص شده است که سرویس AuthService یک تابع به نام Login دارد. این تابع یک LoginRequest دریافت می کند و یک LoginResponse برمی گرداند. سپس ابزارهای gRPC می توانند بر اساس این تعریف، کدهای لازم برای کلاینت و سرور را در زبان های مختلف تولید کنند.

مزیت IDL این است که قرارداد بین کلاینت و سرور شفاف و دقیق می شود. اگر چند تیم مختلف روی بخش های مختلف سیستم کار کنند، همه می دانند که سرویس چه interface ای دارد و چگونه باید با آن ارتباط برقرار کنند. همچنین استفاده از IDL باعث کاهش خطاهای ناسازگاری بین کلاینت و سرور می شود.

خطا های رایج درRPC

در RPC علاوه بر خطا های معمول برنامه، خطاهای مربوط به شبکه و سیستم توزیع شده نیز وجود دارند. یکی از رایج ترین خطا ها timeout است. این خطا زمانی رخ می دهد که کلاینت درخواست را ارسال می کند اما در زمان مشخص پاسخی از سرور دریافت نمی کند. ممکن است سرور کند باشد، شبکه مشکل داشته باشد یا درخواست در مسیر گم شده باشد.

خطای دیگر در دسترس نبودن سرور است. ممکن است سرور خاموش باشد، برنامه سرور crash کرده باشد یا پورت مورد نظر باز نباشد. در این حالت کلاینت نمی تواند با سرور ارتباط برقرار کند.

خطای دیگر مربوط به قطع شدن شبکه یا packet loss است. چون RPC روی شبکه انجام می شود، هر مشکل ارتباطی می تواند باعث شکست درخواست شود.

یکی دیگر از مشکلات مهم، اجرای تکراری درخواست است. فرض می کنیم کلاینت یک درخواست پرداخت یا تغییر داده ارسال می کند، اما پاسخ را دریافت نمی کند. کلاینت ممکن است تصور کند درخواست انجام نشده و دوباره آن را ارسال کند. اگر سرور درخواست اول را انجام داده باشد، اجرای دوباره درخواست می تواند باعث مشکل شود. به همین دلیل در طراحی RPC باید به مدیریت retry توجه کرد.

همچنین ممکن است نسخه کلاینت و سرور با هم ناسازگار شوند. مثلا کلاینت انتظار دارد تابعی با یک نوع ورودی خاص وجود داشته باشد، اما سرور تغییر کرده باشد. استفاده از IDL و versioning می تواند این مشکل را کاهش دهد.

مقایسه RPC باREST

REST و RPC هر دو برای ارتباط بین سرویس ها استفاده می شوند، اما نگاه آن ها متفاوت است. در RPC تمرکز روی فراخوانی عملیات یا تابع است. یعنی کلاینت می گوید یک عملیات خاص را انجام بده، مثلا Login ، GetFileیا Calculate . در REST تمرکز بیشتر روی resource یا منبع است. یعنی سیستم با منابعی مثل user، file یا image کار می کند و با استفاده از متدهای HTTP مانندGET ، POST، PUT و DELETE روی آن ها عملیات انجام می دهد.

در RPC مسیرها معمولا نشان دهنده عملیات هستند. برای مثال:

/AuthService/Login

اما در REST مسیرها معمولا نشان دهنده منابع هستند. برای مثال:

/users
/files/1

در REST برای دریافت داده معمولا ازGET ، برای ایجاد داده از POST ، برای به روز رسانی از PUT یا PATCH و برای حذف از DELETE استفاده می شود. اما در RPC ممکن است همه عملیات ها در قالب یک درخواست POST ارسال شوند و نام تابع داخل پیام یا مسیر مشخص شود.

از نظر سادگی، REST معمولا برای API های عمومی، وب سرویس های ساده و ارتباطات مبتنی بر HTTP انتخاب مناسبی است، چون با مرورگرها، ابزارهای تست و زیرساخت های وب بسیار سازگار است. اما RPC مخصوصا gRPC برای ارتباطات داخلی بین سرویس ها، سیستم های microservice ، نیاز به کارایی بالا و قرارداد دقیق بین کلاینت و سرور مناسب تر است.

مقایسه چند فناوری رایجRPC

فناوری های مختلفی برای پیاده سازی RPC وجود دارند که هر کدام ویژگی ها و کاربردهای خاص خود را دارند. در این بخش چند مورد مهم بررسی می شوند.

gRPC یکی از فناوری های جدید و پرکاربرد RPC است که توسط Google معرفی شده است. این فناوری معمولا از Protocol Buffers برای تعریف پیام ها و سرویس ها استفاده می کند و روی HTTP/2 اجرا می شود gRPC .از چند زبان برنامه نویسی پشتیبانی می کند و برای سیستم های توزیع شده و microservice ها بسیار مناسب است. یکی از مزایای مهم آن پشتیبانی از streaming است. یعنی کلاینت و سرور می توانند علاوه بر درخواست و پاسخ ساده، جریان های داده ای دوطرفه نیز داشته باشند. از نظر کارایی نیز به دلیل استفاده از فرمت باینری Protocol Buffers و HTTP/2 عملکرد بسیار خوبی دارد. البته پیاده سازی آن نسبت به REST یا JSON-RPC کمی پیچیده تر است.

JSON-RPC یک روش ساده تر برای پیاده سازی RPC است که در آن پیام ها با قالب JSON ارسال می شوند. در JSON-RPC معمولا نام متد، پارامترها و شناسه درخواست در پیام قرار می گیرد. چون JSON خوانا و ساده است، پیاده سازی JSON-RPC آسان تر است و برای API های ساده یا سیستم هایی که به پیچیدگی gRPC نیاز ندارند مناسب است. اما از نظر کارایی معمولا نسبت به gRPC ضعیف تر است، چون JSON متنی است و حجم بیشتری نسبت به فرمت های باینری دارد. همچنین JSON-RPC به صورت ذاتی IDL اجباری و streaming پیشرفته ندارد.

XML-RPC یکی از روش های قدیمی تر RPC است که از XML برای نمایش پیام ها استفاده می کند. این فناوری از نظر ساختار ساده است اما به دلیل استفاده ازXML ، پیام ها معمولا حجیم تر و پردازش آن ها سنگین تر است XML-RPC .برای سیستم های قدیمی و legacy مناسب تر است، اما امروزه در بسیاری از سیستم های جدید جای خود را به فناوری هایی مانند gRPC ، RESTیا JSON-RPC داده است.

Apache Thrift نیز یک framework برای ساخت سرویس های چند زبانه است. در Thrift ابتدا interface سرویس ها با یک زبان تعریف مشخص می شود و سپس کدهای کلاینت و سرور برای زبان های مختلف تولید می شوند. Thrift از چند پروتکل و چند نوع transport پشتیبانی می کند و برای سیستم هایی که به ارتباط سریع و چند زبانه نیاز دارند مناسب است. از نظر ایده، Thriftتا حدی شبیه gRPC است، زیرا هر دو از تعریف interface و تولید کد استفاده می کنند، اما تفاوت هایی در ابزارها، فرمت ها و اکوسیستم دارند.

Java RMI یا Java Remote Method Invocation یکی از فناوری های قدیمی و شناخته شده برای پیاده سازی RPC در زبان Java است. ایده اصلی Java RMI این است که یک برنامه Java بتواند متدی را روی یک شی Java که در یک JVM دیگر و حتی روی یک ماشین دیگر قرار دارد، فراخوانی کند. در این حالت، کلاینت با یک شیء راه دور کار می کند و فراخوانی متدهای آن از دید برنامه نویس تا حد زیادی شبیه فراخوانی متدهای عادی در Java است. در Java RMI ابتدا یک interface تعریف می شود که متدهای قابل فراخوانی از راه دور را مشخص می کند. سپس سرور این interface را پیاده سازی می کند و شیء مورد نظر را در RMI Registry ثبت می کند. کلاینت با استفاده از نام ثبت شده در registry به آن شی دسترسی پیدا می کند و متدهای آن را فراخوانی می کند. مزیت اصلی Java RMI این است که برای برنامه های نوشته شده با Java بسیار طبیعی و ساده است، زیرا با مفاهیم شیء گرایی Java هماهنگ است. همچنین برنامه نویس می تواند با object ها، interface ها و exception های Java کار کند. اما محدودیت مهم آن این است که به اکوسیستم Java وابستگی زیادی دارد و برای سیستم هایی که چند زبان برنامه نویسی مختلف دارند، انتخاب مناسبی نیست. به همین دلیل در سیستم های توزیع شده مدرن که سرویس ها ممکن است با زبان های مختلف نوشته شوند، معمولا فناوری هایی مانند gRPC یا REST بیشتر استفاده می شوند. از نظر فرمت داده، Java RMI معمولا از serialization مخصوص Java استفاده می کند. این موضوع باعث می شود ارتباط بین برنامه های Java ساده باشد، اما خوانایی و سازگاری آن با زبان های دیگر کاهش پیدا کند. بنابراین Java RMI برای سیستم های داخلی مبتنی بر Java مناسب است، ولی برای API های عمومی یا سیستم های چند زبانه انتخاب ایده آلی نیست.

SOAP مخفف Simple Object Access Protocol است و یکی از پروتکل های قدیمی و رسمی برای ارتباط بین سرویس ها محسوب می شود. SOAP معمولا از XML برای قالب پیام ها استفاده می کند و می تواند روی پروتکل های مختلفی مانند HTTP منتقل شود. درSOAP ، پیام ها ساختار مشخصی دارند و معمولا شامل بخش هایی مانند Envelope، Headerو Body هستند. یکی از ویژگی های مهم SOAP این است که معمولا همراه با WSDL استفاده می شود. WSDL یا Web Services Description Language فایلی است که interface سرویس را تعریف می کند. در WSDL مشخص می شود که سرویس چه عملیات هایی دارد، ورودی و خروجی هر عملیات چیست و کلاینت چگونه باید با سرویس ارتباط برقرار کند. بنابراین SOAP از نظر قراردادی بودن و رسمی بودن ساختار قدرتمندی دارد. مزیت SOAP این است که استانداردهای زیادی در اطراف آن وجود دارد. برای مثال در سیستم های سازمانی وenterprise ، مواردی مانند امنیت، تراکنش، اعتبارسنجی پیام و قرارداد رسمی سرویس اهمیت زیادی دارند و SOAP برای چنین محیط هایی مدت ها استفاده شده است. به

همین دلیل هنوز هم در برخی سیستم های بانکی، دولتی، بیمه ای و legacy می توان SOAP را مشاهده کرد . اما SOAP معمولا نسبت به روش های جدیدتر سنگین تر است. چون پیام ها با XML نوشته می شوند، حجم بیشتری دارند و پردازش آن ها هزینه بیشتری دارد. همچنین پیاده سازی و تست SOAP معمولا از REST یا JSON-RPC پیچیده تر است. به همین دلیل برای پروژه های ساده، API های سبک و سیستم های مدرن microservice معمولا SOAP انتخاب اول نیست، مگر این که پروژه نیاز به استانداردهای رسمی و سازگاری با سیستم های قدیمی داشته باشد.

جدول مقایسه فناوری های RPC

Use Case	Streaming	IDL	Data Format	Technology
Microservices	دارد	دارد	Protocol Buffers	gRPC
API	معمولا ندارد	معمولا ندارد یا اختیاری است	JSON	JSON-RPC
سیستم های قدیمی و legacy	ندارد	معمولا ندارد	XML	XML-RPC
سیستم های چند زبانه و توزیع شده	محدود و وابسته به پیاده سازی	دارد	Binary / Compact / JSON	Apache Thrift
سیستم های داخلی مبتنی بر Java	ندارد	دارد، با interfaceهای Java	Java Serialization	Java RMI
سیستم های enterprise، بانکی، دولتی و legacy	معمولا ندارد	دارد، معمولا با WSDL	XML	SOAP

پاسخ به پرسش های تحلیلی

1. چرا RPC باعث می شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد؟

RPC با استفاده از stub ها و پنهان کردن جزئیات شبکه باعث می شود برنامه نویس احساس کند در حال فراخوانی یک تابع معمولی است. در سمت کلاینت، Client Stub مانند یک تابع محلی در اختیار برنامه قرار می گیرد. وقتی این تابع صدا زده می شود، Client Stub خودش پارامترها را serialization می کند، درخواست را به سرور می فرستد، پاسخ را دریافت می کند و نتیجه را به برنامه برمی گرداند. بنابراین برنامه نویس لازم نیست مستقیما با socket ، ساخت پیام شبکه ای، ارسال داده و دریافت پاسخ کار کند. همین پنهان سازی باعث می شود فراخوانی راه دور از نظر ظاهری شبیه فراخوانی محلی باشد.

2. چرا این شباهت می تواند خطرناک یا گمراه کننده باشد؟

این شباهت می تواند گمراه کننده باشد، چون فراخوانی راه دور در واقع مانند فراخوانی محلی ساده، سریع و قطعی نیست. در فراخوانی محلی، تابع معمولا در همان فرآیند اجرا می شود و احتمال خطاهای شبکه وجود ندارد. اما درRPC ، درخواست باید از شبکه عبور کند و ممکن است باtimeout ، قطع ارتباط، کندی شبکه، در دسترس نبودن سرور یا خطای serialization مواجه شود. اگر برنامه نویس فراموش کند که این فراخوانی در واقع از راه دور انجام می شود، ممکن است تعداد زیادی RPC کوچک و پشت سر هم طراحی کند که باعث کندی شدید سیستم شود. همچنین ممکن است مدیریت خطا، retry و timeout را جدی نگیرد و سیستم در شرایط واقعی ناپایدار شود.

3. چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

در فراخوانی راه دور خطاهای شبکه ای و توزیع شده وجود دارند که در فراخوانی محلی دیده نمی شوند. برای مثال ممکن است سرور در دسترس نباشد، اتصال شبکه قطع شود، پاسخ دیر برسد و timeout رخ دهد، پیام درخواست یا پاسخ در مسیر از بین برود، سرور درخواست را انجام داده باشد اما پاسخ به کلاینت نرسد، یا به دلیل retryیک عملیات چند بار اجرا شود. همچنین ممکن است نسخه interface در کلاینت و سرور با هم ناسازگار باشد یا داده ها هنگام serialization و deserialization به خطا بخورند. این خطاها نشان می دهند که RPC باید با مدیریت خطای دقیق طراحی شود.

4. در چه شرایطی gRPC انتخاب بهتری از REST است؟

gRPC زمانی انتخاب بهتری از REST است که سیستم از چند سرویس داخلی تشکیل شده باشد و ارتباط بین آن ها زیاد، ساختارمند و حساس به کارایی باشد. برای مثال در معماری microservices ، سرویس ها دائما با هم ارتباط دارند و لازم است درخواست ها سریع، کم حجم و دقیق باشند. در چنین شرایطی gRPC به دلیل استفاده از Protocol Buffers ، پشتیبانی ازHTTP/2 ، تولید خودکار کد از روی فایل proto و داشتن قرارداد مشخص بین کلاینت و سرور انتخاب مناسبی است. همچنین اگر سیستم به streaming نیاز داشته باشد، مثلا سرور بخواهد جریان پیوسته ای از داده را به کلاینت ارسال کند یا ارتباط دوطرفه جریان داده لازم باشد، gRPC نسبت به REST انتخاب قوی تری است.

5. در چه شرایطی REST انتخاب ساده تر یا مناسب تر است؟

REST زمانی مناسب تر است که API ساده، عمومی و مبتنی بر منابع باشد. برای مثال اگر هدف طراحی یک سرویس وب باشد که کاربران یا کلاینت های مختلف از طریق HTTP با آن کار کنند، REST معمولا ساده تر و قابل فهم تر است. REST با ابزارهای رایج وب مانند مرورگر، curl، Postman، reverse proxy ها و سیستم های cache سازگاری بسیار خوبی دارد. همچنین وقتی نیاز به streaming پیچیده، کارایی بسیار بالا یا قرارداد سخت گیرانه بین سرویس ها وجود ندارد، REST می تواند انتخاب ساده تر و عملی تری باشد. برای پروژه های کوچک، API های عمومی و سرویس هایی که بیشتر عملیات CRUD روی منابع انجام می دهند، REST معمولا پیاده سازی و تست آسان تری دارد.

جمع بندی

در این بخش مفهوم RPC و نقش آن در سیستم های توزیع شده بررسی شد. RPC روشی است که اجازه می دهد یک برنامه تابعی را روی یک ماشین یا سرویس دیگر فراخوانی کند، در حالی که این فراخوانی از دید برنامه نویس شبیه یک تابع محلی دیده می شود. برای انجام این کار، اجزایی مانند Client Stub ، Server Stub، serialization، deserialization و در بسیاری از فناوری ها IDL استفاده می شوند.